

Instrucciones Técnicas del Proyecto

OBJETIVO

Desarrollar y desplegar una API REST robusta con Spring Boot y PostgreSQL para gestionar usuarios, eventos académicos, sesiones e inscripciones, aplicando seguridad JWT, autorización por roles, Redis, validación, manejo centralizado de excepciones, documentación OpenAPI, observabilidad y generación de reportes descargables.

TECNOLOGÍAS OBLIGATORIAS

Spring Web, Spring Data JPA, Spring Security, Bean Validation, PostgreSQL, JWT, Spring Data Redis, Bucket4j o equivalente, Springdoc OpenAPI, Spring Boot Actuator, JUnit, Mockito y Docker. Para los reportes se podrá utilizar Apache POI, OpenPDF u otra librería equivalente.

1. ARQUITECTURA GENERAL

Implementar un monolito modular organizado por dominio. Cada módulo deberá separar:

- Controladores
- Servicios e interfaces de servicio
- Repositorios
- Entidades
- DTO de entrada y respuesta
- Mapeadores
- Excepciones específicas

2. MODELO DE DATOS

La base de datos deberá contener como mínimo:

Tabla	Descripción
users	Usuarios, credenciales, estado
roles	Roles ADMIN, ORGANIZER y PARTICIPANT
user_roles	Relación de usuarios y roles
categories	Categorías de eventos
events	Evento, modalidad, cupo, fechas, organizador y estado
sessions	Sesiones y horarios de cada evento
registrations	Inscripciones de participantes
audit_logs	Auditoría de operaciones críticas

La base de datos será proporcionada mediante scripts SQL que contengan la creación de las tablas, relaciones, restricciones e índices necesarios para ejecutar el proyecto.

Antes de iniciar la aplicación, los estudiantes deberán crear la base de datos y ejecutar los scripts entregados sobre una instancia de PostgreSQL. Las entidades JPA deberán construirse a partir de la estructura, relaciones y tipos de datos definidos en dichos scripts.

3. ROLES Y PERMISOS

- **ADMIN:** administra usuarios, roles, categorías, estados y reportes generales.
- **ORGANIZER:** gestiona únicamente sus eventos, sesiones e inscripciones.
- **PARTICIPANT:** consulta eventos, crea y cancela sus propias inscripciones.

Además del rol, se deberá validar la propiedad del recurso. Un organizador no podrá modificar eventos pertenecientes a otro organizador.

4. AUTENTICACIÓN Y AUTORIZACIÓN

- Registro, inicio de sesión, renovación, cierre de sesión y consulta del usuario autenticado.
- Contraseñas cifradas con BCrypt.
- Access token JWT con tiempo de expiración corto.
- Protección de endpoints mediante Spring Security y @PreAuthorize.
- Mensajes de autenticación genéricos para evitar revelar si un correo existe.

5. AUTENTICACIÓN ADICIONAL

- Refresh token con expiración y revocación.
- Protección de endpoints mediante Spring Security y @PreAuthorize.
- Mensajes de autenticación genéricos para evitar revelar si un correo existe.

6. REDIS Y RATE LIMITING

Redis será obligatorio para almacenar información temporal utilizada por los mecanismos de seguridad y rendimiento del sistema. No deberá emplearse como almacenamiento permanente de usuarios, eventos, inscripciones ni otra información principal, la cual deberá persistirse en PostgreSQL.

Redis deberá utilizarse para implementar las siguientes funcionalidades:

- Rate limiting distribuido mediante contadores temporales asociados a una dirección IP o usuario.
- Bloqueo temporal de usuarios o direcciones IP después de varios intentos fallidos.

Las claves deberán utilizar prefijos que identifiquen su finalidad, como `blocked-user:`. Todas las claves temporales deberán tener un TTL definido.

7. LÍMITES DE SOLICITUDES

El rate limiting deberá implementarse utilizando Redis. Antes de procesar una solicitud, la API deberá incrementar de forma atómica el contador asociado a la dirección IP o al usuario autenticado. Cuando se supere el límite configurado, deberá responder con el código 429 Too Many Requests e incluir el encabezado `Retry-After`.

Operación	Identificador	Límite
Inicio de sesión	Dirección IP y correo	5 solicitudes por minuto
Registro	Dirección IP	3 solicitudes por hora
Endpoints públicos	Dirección IP	60 solicitudes por minuto
Endpoints autenticados	Usuario autenticado	120 solicitudes por minuto
Generación de reportes	Usuario autenticado	5 solicitudes por minuto

8. CORS RESTRINGIDO

- Los orígenes permitidos deberán obtenerse desde una variable de entorno.
- Restringir métodos a GET, POST, PUT, PATCH, DELETE y OPTIONS.
- Restringir encabezados a los necesarios, como Authorization y Content-Type.
- No se permite utilizar * como origen en producción.
- No habilitar credenciales si la estrategia de autenticación no lo necesita.

9. REGLAS DE NEGOCIO Y TRANSACCIONES

- No permitir correos ni categorías duplicadas.
- No permitir dos inscripciones del mismo participante en un evento.
- No permitir inscripciones en eventos finalizados o sin cupos.
- Registrar la inscripción y actualizar la disponibilidad dentro de una transacción.
- No eliminar físicamente un evento publicado con inscripciones.
- Aplicar paginación, filtros, búsqueda y ordenamiento en los listados.

10. MANEJO CENTRALIZADO DE EXCEPCIONES

Implementar `@RestControllerAdvice` y respuestas uniformes con fecha, código HTTP, código interno, mensaje y ruta. Se deberán manejar errores de validación por campo y, como mínimo, recursos inexistentes, duplicados, reglas de negocio, acceso prohibido, falta de cupos, tokens inválidos y exceso de solicitudes.

11. SWAGGER Y OPENAPI PROTEGIDOS

- Documentar endpoints, parámetros, DTO, respuestas, códigos HTTP y ejemplos.
- Configurar el esquema Bearer JWT para probar endpoints protegidos.
- Proteger `/swagger-ui/**` y `/v3/api-docs/**` en producción mediante credenciales de evaluación.
- Las credenciales de Swagger deberán configurarse mediante variables de entorno.

12. ACTUATOR Y OBSERVABILIDAD

Incluir la dependencia: `implementation 'org.springframework.boot:spring-boot-starter-actuator'`

Exponer públicamente solo `/actuator/health`, sin detalles internos.

13. REPORTES, ESTADÍSTICAS Y ARCHIVOS DESCARGABLES

El módulo de reportes permitirá consultar indicadores del sistema y descargar archivos generados bajo demanda. Los reportes deberán respetar los roles y la propiedad de los eventos.

Endpoints de estadísticas:

- `GET /api/reports/events/{eventId}/registrations.pdf` - Listado de inscritos en formato PDF (Propietario o ADMIN)
- `GET /api/reports/events/{eventId}/registrations.xlsx` - Listado de inscritos en formato Excel (Propietario o ADMIN)
- `GET /api/registrations/{id}/certificate.pdf` - Comprobante de una inscripción confirmada (Participante propietario)

Los endpoints de estadísticas deberán aceptar filtros por rango de fechas. Los archivos deberán generarse bajo demanda y responder con los encabezados Content-Type y Content-Disposition correctos.

Ejemplo:

```
Content-Type: application/pdf
Content-Disposition: attachment; filename="registrations-event-15.pdf"
```

14. ZONA HORARIA

La API deberá manejar correctamente las zonas horarias. Se recomienda almacenar los instantes en UTC y convertirlos a la zona de negocio al mostrarlos en la interfaz o en los reportes.

Zona de negocio: America/Guayaquil Almacenamiento de instantes: UTC Formato de intercambio: ISO 8601

15. DESPLIEGUE

- Desplegar el backend mediante Docker en Render o una plataforma equivalente.
- Desplegar PostgreSQL y Redis o un servicio compatible.
- Configurar secretos y conexiones mediante variables de entorno.
- Incluir Dockerfile y render.yaml o configuración equivalente.
- Mantener disponibles la API, Swagger y Actuator Health durante la evaluación.

Para Render se deberá considerar como mínimo:

```
server.port=${PORT:8080}
spring.jpa.open-in-view=false
spring.datasource.hikari.maximum-pool-size=5
```

PostgreSQL y Redis deberán ejecutarse como servicios separados. El contenedor de Spring Boot no deberá almacenar archivos permanentes; los reportes se generarán bajo demanda. La memoria de la JVM deberá limitarse mediante JAVA_TOOL_OPTIONS para evitar superar los recursos de la instancia.

16. VARIABLES DE ENTORNO

Todas las credenciales, secretos y configuraciones sensibles deberán definirse mediante variables de entorno. No se permite almacenar información sensible en el código fuente ni en archivos de configuración versionados.

Las variables de entorno mínimas deberán incluir, entre otras:

```
DB_URL
DB_USERNAME
DB_PASSWORD

REDIS_HOST
REDIS_PORT
REDIS_PASSWORD

JWT_SECRET
JWT_ACCESS_EXPIRATION
JWT_REFRESH_EXPIRATION

ALLOWED_ORIGINS
PORT

REDIS_URL
```

17. ENTREGABLES

1. **Repositorio Git:** Código fuente completo, historial de commits de los dos integrantes distribuidos en al menos 3 días diferentes.
2. **Base de datos:** Diagrama entidad-relación, migraciones Flyway y datos iniciales.
3. **Documentación:** README con instalación, variables, ejecución, pruebas y despliegue.
4. **API:** Enlace público del backend y enlace de Swagger UI.
5. **Pruebas:** Evidencias y reporte de ejecución.
6. **Ciente de prueba:** Colección Postman o Bruno.
7. **Video:** Presentación técnica de máximo 10 minutos.
8. **AVAC:** Entregar únicamente el enlace al repositorio remoto.

PARTICIPACIÓN EN GIT

Cada integrante deberá realizar al menos 5 commits funcionales y verificables. Los commits de cada integrante deberán distribuirse en al menos 3 días diferentes. Los mensajes deberán describir el cambio realizado. No se considerarán commits vacíos, cambios de formato usados para aumentar el conteo ni cargas completas realizadas al final.

Si un integrante presenta aportes mínimos, no puede explicar su código o concentra todos sus commits el día de la entrega, se reducirá el criterio de calidad y trabajo colaborativo.

Ese es el contenido técnico exacto. Puedes copiarlo y pegarlo donde necesites.